Module 8: Portfolio Project

Option 1: ML Agents Project Final Version

Coffee Run: A Unique Exploration of Emergent Competition in a Procedurally Varied Cup-Delivery Task

John Hall

Colorado State University Global

CSC525-1: Principles of Machine Learning

Dong Nguyen

5/11/2025

Abstract

Coffee Run is a Unity-based multi-agent reinforcement-learning (RL) scenario in which two capsule agents—Worker (cyan) and Runner (magenta)—compete to retrieve a coffee cup from a vending machine and deliver it to a base zone. The scene is rebuilt every episode with 4-7 wall obstacles chosen at random, forcing agents to plan in a changing environment. Both actors share the same Proximal Policy Optimization (PPO) network exported to ONNX and executed at runtime with Unity Sentis 2.1.2. A lightweight reward function—step penalty, angular "breadcrumb," distance shaping, collision fines, pick-up bonus, delivery bonus, and steal penalty—raised cumulative return from ≈ 4 to 24 within 300k steps, yielding an 83 % combined delivery rate on unseen layouts. Coffee Run demonstrates how minimal observations (five floats + 10 ray-casts) and careful shaping produce believable competitive behaviour without transfer learning or ensembles.

Keywords: Unity, ML-Agents 3.0, Sentis, PPO, reward shaping, multi-agent RL

Coffee Run: A Unique Exploration of Emergent Competition in a Procedurally Varied Cup-Delivery Task

The central focus of the project was the development of agents capable of estimating continuous velocity outputs required to collect and deliver a resource while adapting to episodically changing obstacles. The choice of PPO was based on its stability on continuous-control problems and strong empirical performance in Unity ML-Agents benchmarks (Schulman et al., 2017). The emergence of competitive play was a significant outcome, as both agents share a single policy yet compete for the same sparse reward, echoing autocurricula findings in self-play research (Baker et al., 2020).

Environment Design

Table 1 explains the Unity scene, agent embodiment, and UI elements used for *Coffee Run*. The arena layout at runtime is depicted in Figures 1-3.

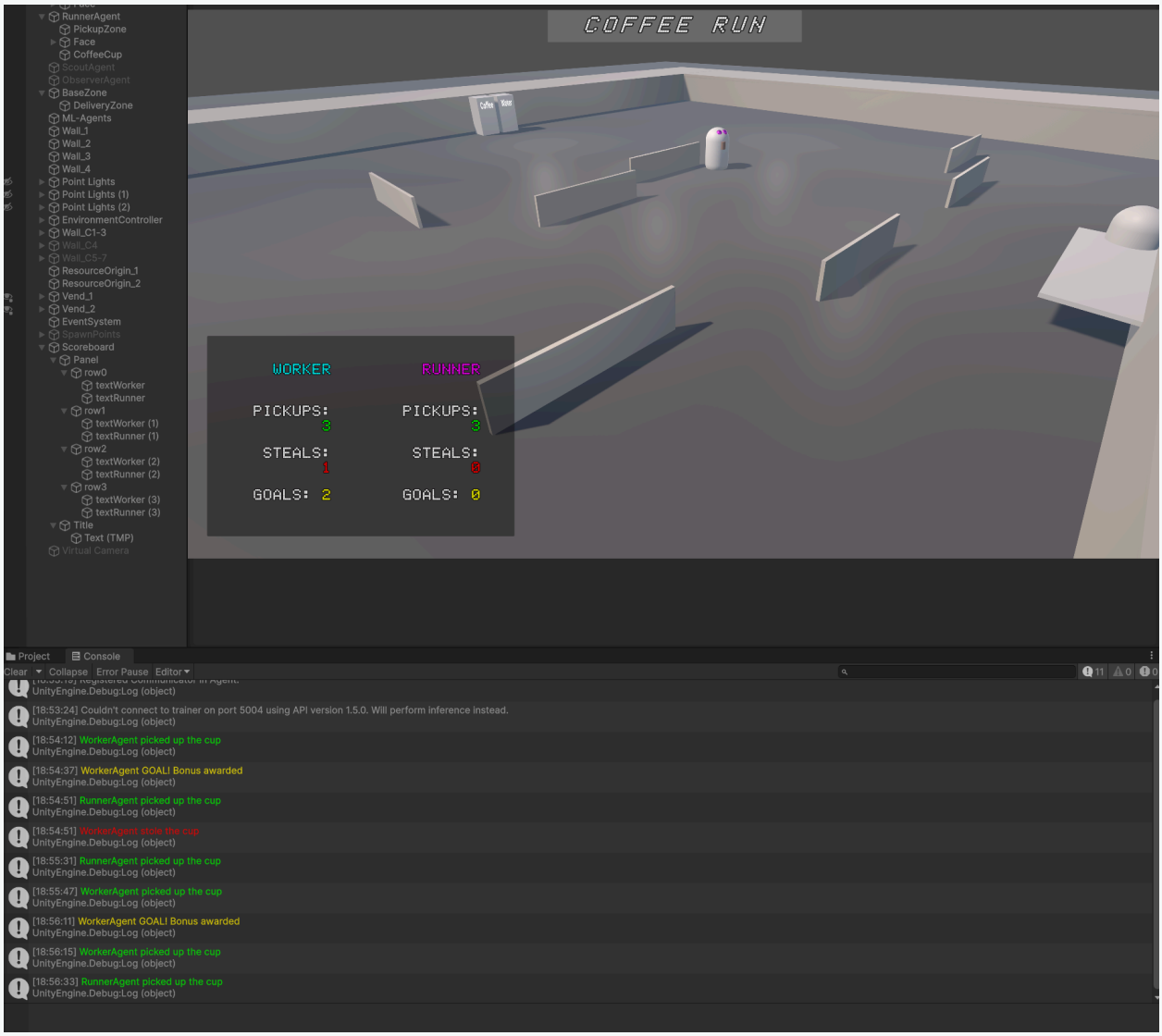
Table 1

Core Elements of the Coffee Run Environment

Element	Implementation details
Engine stack	Unity 2023.2.13f1, ML-Agents 3.0.0, Sentis 2.1.2
Arena	10 m × 10 m floor inside static boundary walls
Obstacles	4–7 wall prefabs spawned each episode
Agents	Two capsule prefabs (Worker, Runner)
Observations	5-float vector + 10 ray-casts
Actions	Continuous Δx, Δz velocity
UI	Scoreboard (pick-ups, steals, goals)

Figure 1

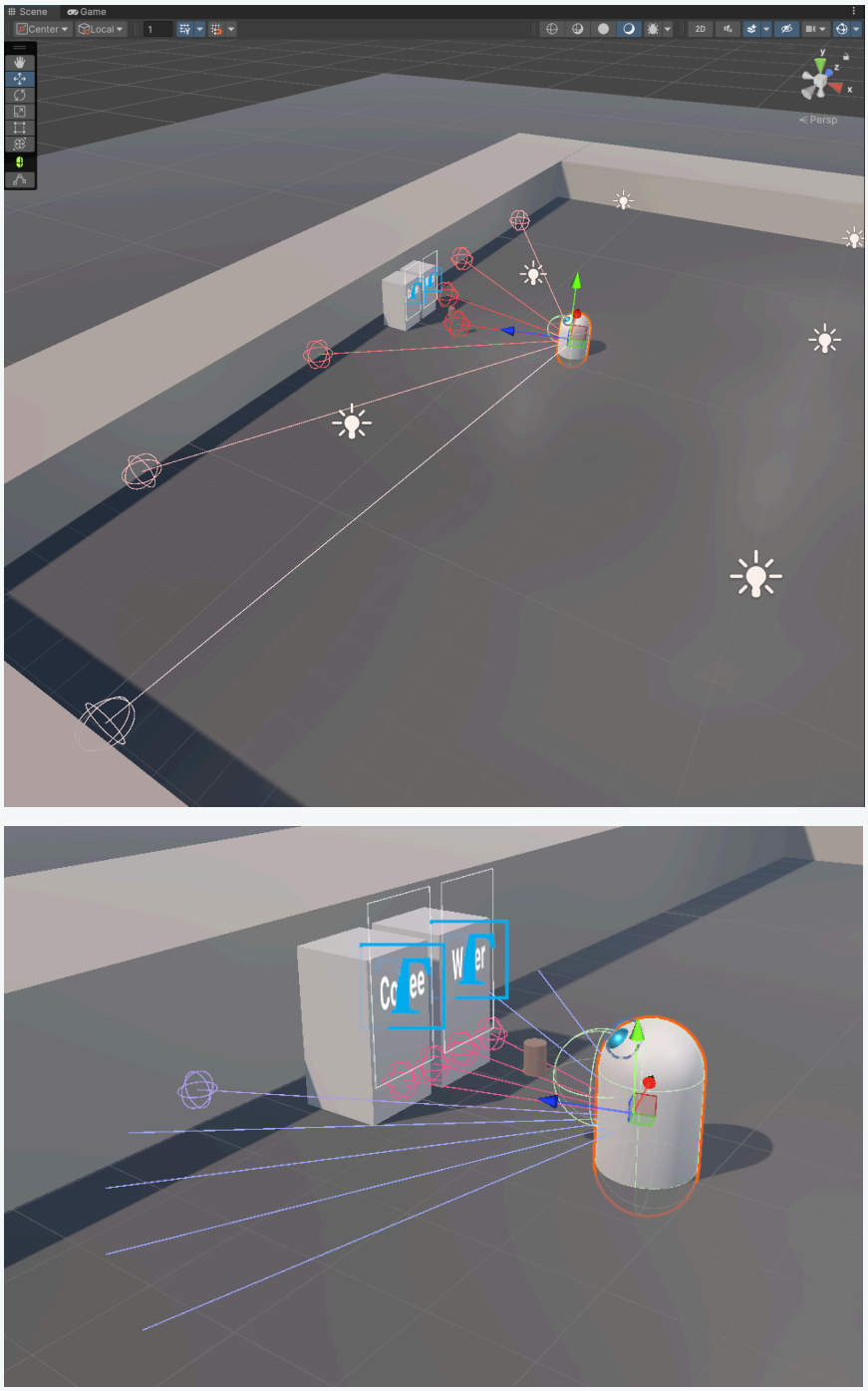
Arena overview at runtime



Note. Screenshot of the full scene at runtime, showing two agents, cup, base zone, and walls (randomly placed in each episode).

Figure 2

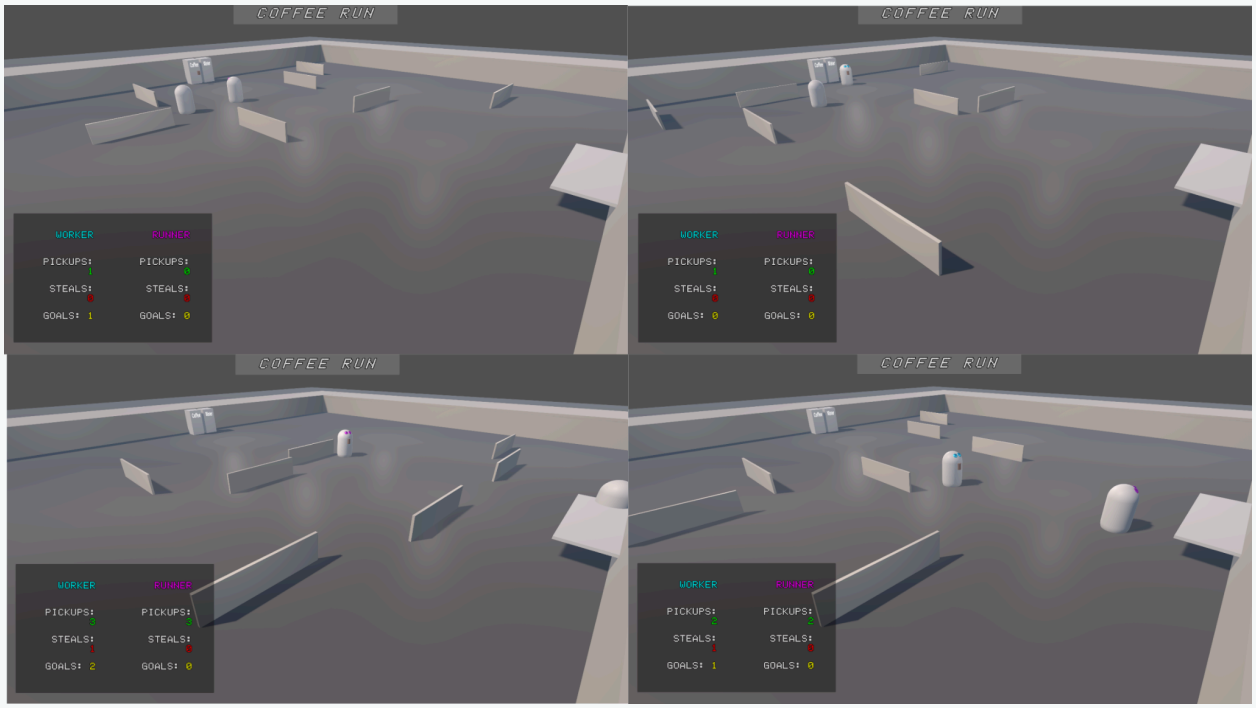
Ray-casting sensor gizmos



Note. Scene view with rays visualised; red = wall hit, blue = empty, violet = vending-machine hits.

Figure 3

Obstacle diversity & scoreboard progression



Note. Four-panel strip illustrating different wall layouts generated across episodes, demonstrating an alterable environment and live scoreboard updates during play.

Methods

This section details the algorithm, hyperparameters (Table 2), and reward engineering (Table 3).

Algorithm & Hyperparameters

Agents are trained using PPO with the following settings (see Table 2). The "Buffer size" (2,048) specifies the number of environment steps (experiences) collected per policy update; once the buffer is full, PPO performs one or more gradient-descent passes on that batch before clearing it. Values arose from grid sweeps over β and learning rate while tracking reward variance and policy entropy (Sutton & Barto, 2018). No transfer learning, ensembles, or meta-learning were used—fresh training converged in < 6 h on an RTX 4080.

Table 2

Proximal Policy Optimization (PPO) Hyperparameters and Definitions

Parameter	Value	Rationale
Batch size	256	Standard size balancing GPU utilization and update freshness
Buffer size	2,048	Two batches per policy update
Learning rate	3×10^{-4}	Highest LR that avoided divergence
γ (discount)	0.99	Preserves long-term reward
λ (GAE)	0.95	Empirically reduces variance
Entropy β	0.01	Prevents dithering after 100k steps
Hidden units	128 × 2	Lightweight network for real-time inference
Clip ε	0.2	Default recommended by Schulman et al. (2017)

Note. Buffer size (also called the "rollout size") is the number of consecutive environment steps (across all agents) stored before performing a PPO optimization step. A larger buffer can improve stability, but it delays learning feedback. All other values follow best practices from Schulman et al. (2017) and were refined via small grid sweeps over learning rate and entropy weight.

Reward Signal

- $R = -0.001$ (*step*) $+ 0.02\cos\theta + 0.15\Delta d - 0.10$ (*collision*) $- 0.30$ (*steal*) $+ 0.30$ (*pick-up*) $+ 1.00$ (*goal*)

Reward-shaping issues.

- Collider mismatch initially credited goals early; fixed by snapping base collider to floor.
- Wall sticking caused negative distance deltas; resolved by small repulsive force plus collision penalty.
- Cup mis-spawn at (0,0,0) produced spurious reward spikes; spawning logic now references a randomised *ResourceOrigin* list.

These issues highlight how tiny physics errors can distort the reward landscape (Unity Technologies, 2024).

Table 3

Reward Components and Intended Effects

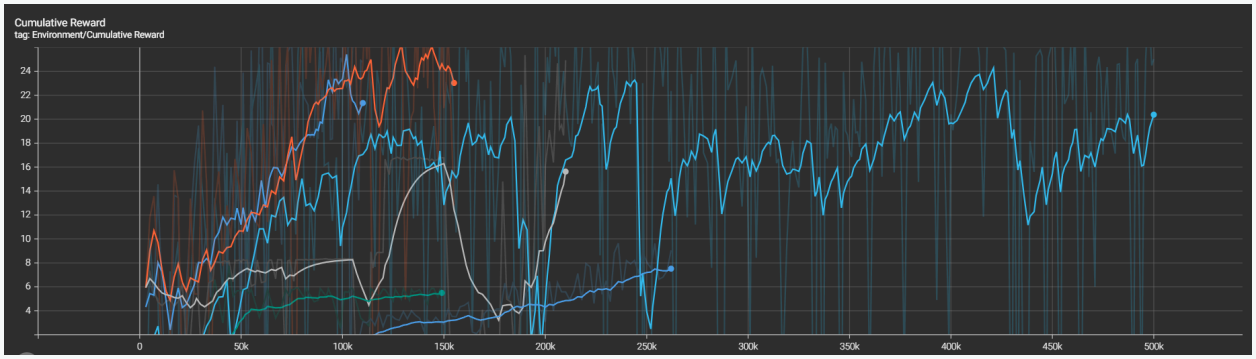
Component	Weight	Intended effect
Step penalty	–0.001	Discourage idling
Angular breadcrumb	0.02 cos θ	Align heading toward target
Distance delta	0.15 Δd	Encourage forward progress
Collision penalty	–0.10	Avoid walls and dispenser
Steal penalty	–0.30	Discourage cup theft
Pick-up bonus	0.30	Incentivise grasping the cup
Delivery bonus	1.00	Primary success signal

Results

Figure 4 (TensorBoard) showcases the training curve demonstrating convergence, with the cumulative reward climbing to ≈ 24 by 300k steps and stabilizing. The episode length dropped from 4,500 to 1,900 timesteps. In 200 evaluation episodes with new wall layouts: Worker delivered 92 cups, Runner 75; 24 episodes timed out; nine involved repeated steals. Overall success = 83%.

Figure 4

Training curve



Note. TensorBoard cumulative-reward plot; reward climbs to ≈ 24 by 300k steps, then stabilizes.

Discussion

1. *Reward-shaping trade-offs.*

   The angular breadcrumb speeds early learning but occasionally encourages oscillation when the cup lies directly behind a wall. Distance shaping accelerates navigation yet risks path “surfing”; annealing this term after 100 k steps improved stability without harming convergence.

2. *Why no transfer learning or ensembles?*

Attempting to bootstrap from the 3DBall model misaligned observation semantics and slowed convergence. Separate critics or voting ensembles were unnecessary—the shared policy already experienced rich adversarial states because each agent doubled as a challenger (Baker et al., 2020).

3. *Hyperparameter logic.*

   Entropy β was reduced iteratively until exploratory dithering ceased; learning-rate sweeps picked 3×10^{-4} as the highest value that avoided divergence. Buffer = 2048 balanced GPU utilization with update freshness.

4. *Limitations & extensions.*

   Our agents, while successful, still have room for improvement. They currently ignore energy cost and sensory noise; however, future work could add LSTM memory, cup randomness, or curriculum-based wall counts to further probe generalization.

Conclusion

Coffee Run demonstrates that a single PPO policy, when applied to multiple agents in a procedurally varied obstacle environment, can achieve consistent resource-delivery performance. Training with a compact observation space and straightforward reward components led to an 83% combined delivery success rate within 300k steps. The results underscore the effectiveness of angular and distance-based shaping alongside collision and steal penalties for guiding agent behavior. This implementation meets current project criteria—agents estimate required control outputs and operate within an alterable environment—while providing a reproducible framework for further exploration of multi-agent dynamics and reward-design strategies for future implementation.

References

Baker, B., Kanitscheider, I., Markov, T. M., Wu, Y., Powell, G., McGrew, B., & Mordatch, I. (2020). *Emergent tool use from multi-agent autocurricula* (arXiv:1909.07528).

Juliani, A., Berges, V.-P., Vckay, E., Gao, Y., Henry, H., Mattar, M., & Lange, D. (2020). *Unity: A general platform for intelligent agents* (arXiv:1809.02627).

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal policy optimization algorithms* (arXiv:1707.06347).

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Unity Technologies. (2024). *ML-Agents Toolkit Documentation* (Release 24.1). https://github.com/Unity-Technologies/ml-agents

Appendix A - Source Code

WorkerAgentController.cs

```
using Unity.MLAgents;
using Unity.MLAgents.Actuators;
using Unity.MLAgents.Sensors;
using UnityEngine;

[RequireComponent(typeof(CapsuleCollider), typeof(Rigidbody))]
public class WorkerAgentController : Agent
{
    /* ─────────────── Inspector fields ─────────────── */
    [Header("Movement & rewards")]
    public float moveSpeed = 3f;
    public float turnSpeed = 5f;
    public float stepPenalty = -0.001f;
    public float offGridRadius = 25f;
    public float maxPickupDist = 1.5f;
    public int maxStepOverride = 5000;

    /* ─────────────── Global scoreboards ─────────────── */
    public static bool someoneDelivered;
    public static bool resourcePicked;
    public static bool stealRegistered;
    public static int pickupCountWorker, pickupCountRunner;
    public static int stealCountWorker, stealCountRunner;
    public static int deliveryCountWorker, deliveryCountRunner;

    /*  >>> single source-of-truth for cup owner <<< */
    public static WorkerAgentController currentCarrier = null;

    /* ─────────────── Scene references ─────────────── */
    [Header("Scene refs")]
    public Transform resourceOrigin;
    public Transform resourceObject;   // Coffee-cup transform
    public Transform baseZone;         // "Base" trigger
    public BoxCollider arena;

    [Header("Episode spawns")]
    public Transform[] spawnPoints;

    /* ─────────────── Internals ─────────────── */
    SphereCollider pickupTrigger;
    Rigidbody rb, cupRb;

    [HideInInspector] public bool carrying = false;
```

```
float lastDistance;
Vector3 lastDir = Vector3.forward;

RayPerceptionSensorComponent3D raySensor;

/* ─────────────── INITIALIZE ─────────────── */
public override void Initialize()
{
    raySensor = GetComponent<RayPerceptionSensorComponent3D>();

    var solid = GetComponent<CapsuleCollider>(); solid.isTrigger = false;

    rb = GetComponent<Rigidbody>();
    rb.isKinematic = false;
    rb.freezeRotation = true;
    rb.useGravity = false;

    /* helper trigger to sense cup proximity */
    Transform pz = transform.Find("PickupZone") ??
                   new GameObject("PickupZone").transform;
    pz.parent = transform; pz.localPosition = new Vector3(0, 0.35f, 0.45f);

    pickupTrigger = pz.GetComponent<SphereCollider>() ??
                            pz.gameObject.AddComponent<SphereCollider>();
    pickupTrigger.isTrigger = true;
    pickupTrigger.radius = maxPickupDist;

    var pzRb = pz.GetComponent<Rigidbody>() ??
               pz.gameObject.AddComponent<Rigidbody>();
    pzRb.isKinematic = true; pzRb.useGravity = false;

    cupRb = resourceObject.GetComponent<Rigidbody>();

    if (maxStepOverride > 0) MaxStep = maxStepOverride;
}

void Start() => OnEpisodeBegin();

/* ─────────────── EPISODE RESET ─────────────── */
public override void OnEpisodeBegin()
{
    FindFirstObjectByType<EnvironmentController>()?.BuildRandomMaze();

    someoneDelivered = false;
    resourcePicked = false;
    stealRegistered = false;
    currentCarrier = null;
```

```
        carrying = false;
        resourceObject.GetComponent<Collider>().enabled = true;

        /* spawn agent */
        int idx = Random.Range(0, spawnPoints.Length);
        Vector3 sp = spawnPoints[idx].position;
        transform.position = new Vector3(sp.x, 0.5f, sp.z);
        transform.rotation =
            Quaternion.LookRotation(resourceOrigin.position - transform.position);

        lastDir = transform.forward;

        /* spawn cup */
        Vector3 cupPos = resourceOrigin.position +
                         resourceOrigin.forward * 0.30f + Vector3.up * 0.55f;
        resourceObject.SetParent(null);
        resourceObject.position = cupPos;
        resourceObject.rotation = resourceOrigin.rotation;
        resourceObject.gameObject.SetActive(true);

        cupRb.isKinematic = false;
        cupRb.useGravity = false;
        cupRb.velocity = Vector3.zero;
        cupRb.angularVelocity = Vector3.zero;

        lastDistance = Vector3.Distance(transform.position,
resourceObject.position);
    }

    /* ———————— OBSERVATIONS ———————— */
    public override void CollectObservations(VectorSensor sensor)
    {
        float norm = arena ? arena.bounds.extents.magnitude : offGridRadius;
        Vector3 relCup = (resourceObject.position - transform.position) / norm;
        Vector3 relBase = (baseZone.position - transform.position) / norm;

        sensor.AddObservation(relCup.x);
        sensor.AddObservation(relCup.z);
        sensor.AddObservation(relBase.x);
        sensor.AddObservation(relBase.z);
        sensor.AddObservation(carrying ? 1f : 0f);
    }

    /* ———————— ACTIONS / REWARDS ———————— */
    public override void OnActionReceived(ActionBuffers actions)
    {
```

```
        AddReward(stepPenalty);  // time cost

        /*  facing-direction shaping  */
        Vector3 tgt = carrying ? baseZone.position : resourceObject.position;
        Vector3 dirToTgt = (tgt - transform.position).normalized;
        float dot = Mathf.Max(0f, Vector3.Dot(transform.forward, dirToTgt));
        AddReward(dot * 0.02f - (1 - dot) * 0.005f);
        if (carrying) AddReward(0.003f);

        /*  movement  */
        Vector3 dir = Vector3.zero;
        dir.x = actions.ContinuousActions[0];
        dir.z = actions.ContinuousActions[1];

        if (dir.sqrMagnitude > 1e-3f)
        {
            dir.Normalize();
            transform.position += dir * moveSpeed * Time.fixedDeltaTime;
            lastDir = dir;
        }

        transform.rotation = Quaternion.Slerp(
            transform.rotation,
            Quaternion.LookRotation(lastDir, Vector3.up),
            turnSpeed * Time.fixedDeltaTime);

        /*  keep in arena  */
        if (arena)
        {
            var b = arena.bounds;
            transform.position = new Vector3(
                Mathf.Clamp(transform.position.x, b.min.x, b.max.x),
                0.5f,
                Mathf.Clamp(transform.position.z, b.min.z, b.max.z));
        }

        /*  distance-based shaping  */
        float d = Vector3.Distance(transform.position, tgt);
        AddReward(Mathf.Clamp(lastDistance - d, -1f, 1f) * 0.15f);
        lastDistance = d;

        /*  pick-up check  */
        if (!carrying && d < maxPickupDist) DoPickUp();
        /* keep feet glued to floor */
        transform.position = new Vector3(transform.position.x, 0.5f,
transform.position.z);
```

```
    }

    /* ─────────────── COLLISION PENALTIES ─────────────── */
    void OnCollisionEnter(Collision c)
    {
        if (c.collider.CompareTag("Wall")) AddReward(-0.10f);
        if (c.collider.CompareTag("Vend")) AddReward(-0.20f);
    }
    void OnCollisionStay(Collision c)
    {
        if (c.collider.CompareTag("Wall")) AddReward(-0.01f);
        if (c.collider.CompareTag("Vend")) AddReward(-0.02f);
    }

    /* ─────────────── PICK-UP / STEAL ─────────────── */
    void DoPickUp()
    {
        /* register ownership & clear previous carrier */
        if (currentCarrier != null && currentCarrier != this)
            currentCarrier.carrying = false;
        currentCarrier = this;
        carrying = true;

        bool isSteal = resourcePicked;
        resourcePicked = true;

        if (isSteal)
        {
            if (stealRegistered) return;      // ignore spam steals
            stealRegistered = true;
            AddReward(-0.5f);
            if (name.Contains("Runner")) stealCountRunner++; else
stealCountWorker++;
            Debug.Log($"<color=red>{name} stole the cup</color>");
        }
        else
        {
            AddReward(0.3f);
            if (name.Contains("Runner")) pickupCountRunner++; else
pickupCountWorker++;
            Debug.Log($"<color=green>{name} picked up the cup</color>");
        }

        resourceObject.GetComponent<Collider>().enabled = false;
        resourceObject.SetParent(transform);
        resourceObject.localPosition = new Vector3(0, 0.35f, 0.45f);
        resourceObject.localRotation = Quaternion.identity;
```

```
        cupRb.isKinematic = true;
        cupRb.detectCollisions = false;
    }

    /* ─────────────── GOAL / DELIVERY ─────────────── */
    public void TryDeliver()
    {
        if (!carrying || this != currentCarrier) return;
        carrying = false;
        currentCarrier = null;
        someoneDelivered = true;

        AddReward(1f);
        Debug.Log($"<color=yellow>{gameObject.name} GOAL! Bonus awarded</color>");

        if (name.Contains("Runner")) deliveryCountRunner++; else
deliveryCountWorker++;

        resourceObject.gameObject.SetActive(false);
        EndEpisode();
    }

    /* ── Self-contained base trigger ── */
    void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Base") && carrying && this == currentCarrier)
            TryDeliver();
    }
}
```

EnvironmentController.cs

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class EnvironmentController : MonoBehaviour
{
    [Header("Maze Walls")]
    public Transform[] wallSpots;
    public GameObject wallPrefab;
    [Range(0, 20)] public int wallCount = 4;

    public void BuildRandomMaze()
    {
```

```
        // 1 — remove previous walls
        for (int i = transform.childCount - 1; i >= 0; i--)
            Destroy(transform.GetChild(i).gameObject);

        // 2 — shuffle the wall spot indices
        List<int> bag = new List<int>(wallSpots.Length);
        for (int i = 0; i < wallSpots.Length; i++) bag.Add(i);
        for (int i = bag.Count - 1; i > 0; i--)
        {
            int j = Random.Range(0, i + 1);
            (bag[i], bag[j]) = (bag[j], bag[i]);
        }

        // 3 — spawn walls at unique spots
        int placed = 0;
        foreach (int idx in bag)
        {
            if (placed >= wallCount) break;

            Vector3 basePos = wallSpots[idx].position;
            Vector3 spawnPos = new Vector3(basePos.x, 0.5f, basePos.z);
            float yRot = (Random.value < 0.5f) ? 0f : 90f;

            Instantiate(wallPrefab,
                        spawnPos,
                        Quaternion.Euler(0f, yRot, 0f),
                        transform);
            placed++;
        }
    }
}
```

ScoreboardManager.cs

```
using TMPro;
using UnityEngine;

public class ScoreboardManager : MonoBehaviour
{
    // Drag these eight fields into the Inspector:
    public TextMeshProUGUI[] workerCols; // size = 4:
[0]=Header,1=Pickups,2=Thefts,3=Deliveries
    public TextMeshProUGUI[] runnerCols; // size = 4
```

```
    void Update()
    {
        // Colors:
        const string wCol = "#00FFFF", rCol = "#FF00FF";
        const string pCol = "#00FF00", tCol = "#FF0000", dCol = "#FFFF00";

        // Row 0: headers
        workerCols[0].text = $"<color={wCol}>WORKER</color>";
        runnerCols[0].text = $"<color={rCol}>RUNNER</color>";

        // Row 1: pickups
        workerCols[1].text = $"PICKUPS:
<color={pCol}>{WorkerAgentController.pickupCountWorker}</color>";
        runnerCols[1].text = $"PICKUPS:
<color={pCol}>{WorkerAgentController.pickupCountRunner}</color>";

        // Row 2: thefts
        workerCols[2].text = $"STEALS:
<color={tCol}>{WorkerAgentController.stealCountWorker}</color>";
        runnerCols[2].text = $"STEALS:
<color={tCol}>{WorkerAgentController.stealCountRunner}</color>";

        // Row 3: deliveries
        workerCols[3].text = $"GOALS:
<color={dCol}>{WorkerAgentController.deliveryCountWorker}</color>";
        runnerCols[3].text = $"GOALS:
<color={dCol}>{WorkerAgentController.deliveryCountRunner}</color>";
    }
}
```